

**ОПИСАНИЕ И АНАЛИЗ
АРХИТЕКТУРЫ НЕЙРОННОЙ СЕТИ
D&D BATTLE AI**

Копьёв Иван Андреевич

Графовая actor-critic модель с обучением методом PPO

Рязань, 2026

1 Назначение нейросетевой модели

Нейросетевая модель D&D Battle AI предназначена для выбора действий участников пошагового тактического боя в дискретной клеточной среде. В отличие от задач классификации, где сеть формирует единственный ответ по фиксированному входному объекту, рассматриваемая модель функционирует как агент обучения с подкреплением. На каждом шаге она получает состояние боя, выбирает составное действие, после чего боевой движок изменяет состояние и вычисляет скалярную награду.

Основной используемой архитектурой является GNNPPOActorCritic: графовый кодировщик состояния объединён с actor-critic моделью, оптимизируемой алгоритмом Proximal Policy Optimization. Сохранённая в проекте полносвязная MLP-модель служит средством обратной совместимости и базовым вариантом для экспериментов, но не является основной моделью боевого искусственного интеллекта.

2 Математическая постановка задачи

Бой представляется конечным марковским процессом принятия решений. В момент времени t среда находится в состоянии s_t , агент получает наблюдение o_t , выбирает действие a_t согласно стохастической политике $\pi\theta(a_t|o_t)$, получает награду r_t и переходит в состояние s_{t+1} . Параметры θ должны максимизировать ожидаемую дисконтированную сумму наград.

$$J(\theta) = E\pi\theta [\sum_t \gamma^t r_t], \quad \gamma = 0,99.$$

Полное состояние включает характеристики всех существ, карту, инициативу, ресурсы, условия и историю текущего хода. Наблюдение строится относительно активного актёра, поэтому одна и та же боевая ситуация кодируется по-разному для разных участников. Такой подход соответствует частично наблюдаемой постановке: скрытые существа и недоступные по линии видимости цели не должны предоставлять агенту недопустимую информацию.

Действие имеет иерархическую структуру. Сначала определяется категория действия, затем его конкретный вид, цель, клетка перемещения, заклинание, уровень ячейки, предмет либо дополнительный параметр. Поэтому политика представлена набором связанных категориальных распределений, а не одним плоским выходным вектором.

3 Представление входного состояния

3.1 Сущностное наблюдение

Для основной модели используется entity-based представление. Оно сохраняет разделение между активным существом, прочими участниками, локальной картой и глобальным состоянием. Это позволяет обрабатывать переменное число существ и формировать отношения между ними непосредственно внутри графового кодировщика.

Компоне нт	Разм ерность	Содержание
actor_features	128	Параметры активного существа, экономика действий, класс, раса, роль, ресурсы, заклинания, предметы, состояния и тактические признаки.
entities_features	8 × 90	До четырёх ближайших союзников и четырёх ближайших противников. Отсутствующие сущности дополняются нулями.
entity_mask	8	Исключает дополненные позиции из message passing и pooling.
map_features	25 × 10 = 250	Окно 5 × 5 вокруг актёра: тип местности, препятствия, укрытия, стоимость движения, достижимость и видимость.
global_features	8	Раунд, позиция в инициативе, размер карты, команда активного существа и число оставшихся союзников/врагов.

3.2 Признаки актёра и других существ

Признаки актёра включают нормализованный уровень и бонус мастерства, идентификаторы класса, подкласса, расы и боевой роли, флаги черт, текущие action, bonus action и reaction, оставшееся перемещение, ресурсы Second Wind, Action Surge, Channel Divinity и Arcane Recovery, текущие и максимальные ячейки заклинаний уровней 1–3, подготовленные заклинания, доступные предметы, укрытие и число видимых противников.

Для каждой другой сущности кодируются отношение к команде, координаты, доля здоровья, класс брони, роль, challenge rating, оценка опыта, состояния prone, grappled, hidden и dodging, наличие реакции и концентрации, сопротивления, иммунитеты и уязвимости, тактическая угроза, расстояние, линия видимости, укрытие и достижимость активным существом.

4 Графовое представление боя

4.1 Узлы и рёбра

Каждый участник боя представляется узлом графа. Активный актёр преобразуется из специализированного 128-мерного описания в общую 90-мерную схему узла. После добавления до восьми сущностей максимальное число узлов равно девяти. Начальное состояние каждого узла проецируется линейным слоем $90 \rightarrow 64$ с функцией активации ReLU.

Граф является плотным ориентированным: признаки вычисляются для каждой допустимой пары узлов. Ребро содержит семь значений: расстояние, принадлежность к одной команде, отношение противников, возможность атаки, возможность помощи, наличие линии видимости и величину укрытия. Вектор ребра проецируется слоем $7 \rightarrow 64$.

Признак ребра	Назначение
---------------	------------

distance	Манхэттенское расстояние между существами.
same_team	Флаг принадлежности к одной команде.
enemy_relation	Флаг противостоящих команд.
can_attack	Наличие допустимой ближней или дальней атаки с учётом расстояния и видимости.
can_help	Возможность оказать помощь союзнику либо помочь против выбранной цели.
line_of_sight	Доступность прямой видимости.
cover_between	Уровень укрытия между участниками.

4.2 Передача сообщений

Кодировщик выполняет два шага передачи сообщений. Для направленной пары узлов i и j сообщение вычисляется из текущих скрытых состояний источника и цели, а также признаков ребра. Конкатенированный вектор размерности 192 обрабатывается многослойным перцептроном $192 \rightarrow 64 \rightarrow 64$.

$$m_{ij} = MLPmsg([h_i, h_j, e_{ij}]).$$

Входящие сообщения усредняются с учётом pair mask. Обновление узла выполняется по его предыдущему состоянию и агрегированному сообщению с помощью MLP $128 \rightarrow 64 \rightarrow 64$.

$$\bar{m}_j = (1 / |N(j)|) \sum_i m_{ij}, \quad h'_j = MLPupd([h_j, \bar{m}_j]).$$

Маска сущностей применяется до и после message passing, поэтому фиктивные узлы padding не создают сообщений и не влияют на итоговые представления.

4.3 Агрегация графа

После двух шагов передачи сообщений формируются четыре 64-мерных представления: активного актёра, среднего состояния союзников, среднего состояния противников и среднего состояния всего боя. Раздельный pooling

союзников и противников важен, поскольку одинаковые локальные характеристики имеют разный смысл в зависимости от принадлежности к команде.

5 Архитектура GNN PPO Actor-Critic

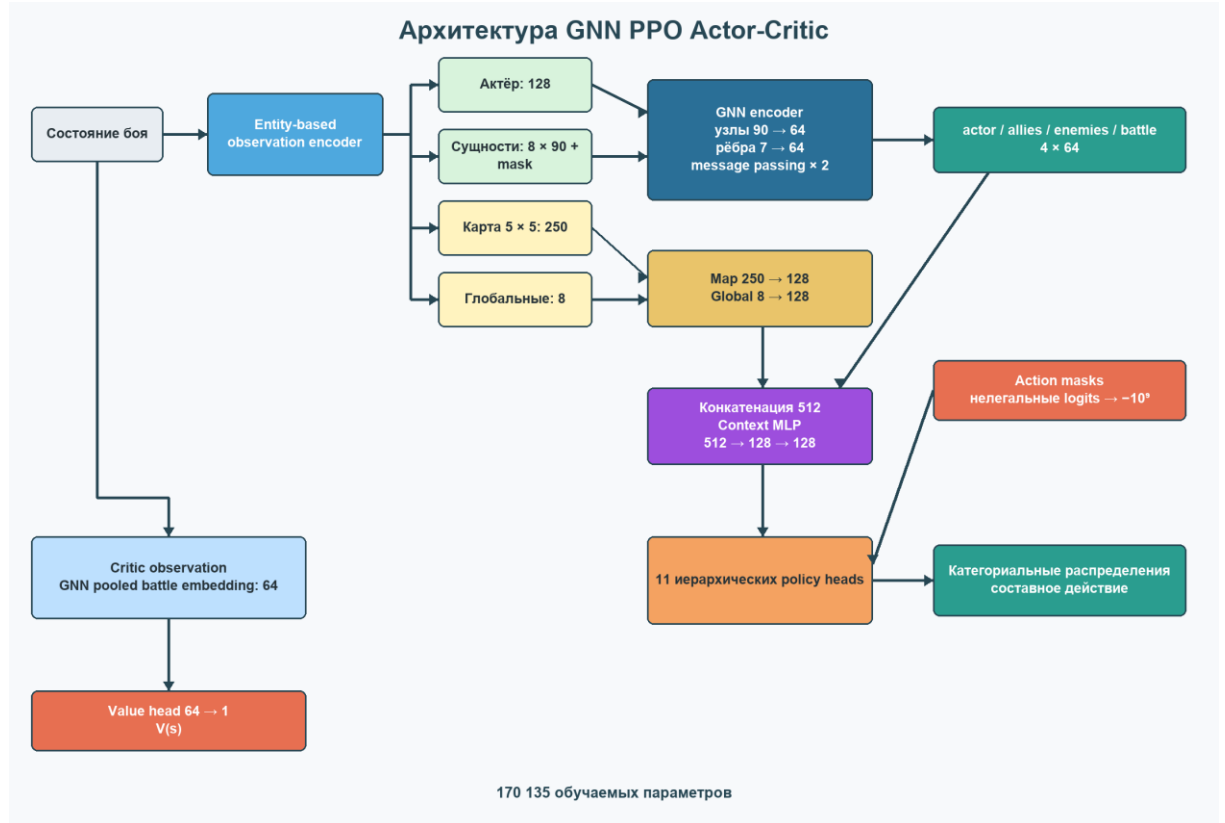


Рисунок 1 – Структурная схема архитектуры GNN PPO Actor-Critic

5.1 Формирование общего контекста

Локальная карта независимо проецируется полносвязным слоем $250 \rightarrow 128$, а глобальные признаки – слоем $8 \rightarrow 128$. Оба слоя используют ReLU. Четыре GNN embedding размерности 64 и два проекционных embedding размерности 128 конкатенируются.

$$z = [h_{actor}, h_{allies}, h_{enemies}, h_{battle}, h_{map}, h_{global}] \in R^{512}.$$

Общий вектор передаётся в context encoder $512 \rightarrow 128 \rightarrow 128$. Полученное 128-мерное представление используется всеми выходными головами политики.

Такая организация позволяет совместно учитывать личные ресурсы актёра, отношения между существами и пространственную обстановку.

5.2 Иерархические выходные головы политики

Голова	Число выходов	Назначение
action_category_head	6	Категория: основное, бонусное, перемещение, реакция, конец хода, классовая способность.
main_action_type_head	11	Конкретный вид основного действия.
bonus_action_type_head	8	Тип бонусного действия.
reaction_type_head	8	Тип реакции.
class_feature_head	8	Активируемая классовая способность.
target_head	8	Цель среди закодированных существ.
move_head	64	Клетка назначения в дискретном пространстве перемещения.
spell_head	13	Заклинание из поддерживаемого набора.
slot_level_head	4	Уровень ячейки 0–3.
item_head	4	Предмет инвентаря.
option_head	16	Оружие, вариант действия или иной дополнительный параметр.

Каждая голова формирует logits отдельного категориального распределения. Итоговая логарифмическая вероятность составного действия равна сумме $\log \text{probability}$ выбранных компонентов. Аналогично суммируется энтропия, применяемая для поддержания исследования пространства действий.

$$\log \pi_{\theta}(a/s) = \sum_k \log \pi_{\theta,k}(a_k/s), \quad H(\pi) = \sum_k H(\pi_k).$$

5.3 Маскирование недопустимых действий

Перед построением категориальных распределений к каждому выходу применяется логическая маска, сформированная боевым движком. Она учитывает экономику действий, уровень, классовые и расовые особенности, ресурсы, ячейки заклинаний, инвентарь, дальность, препятствия, стоимость местности, линию видимости и укрытия. Logits запрещённых вариантов заменяются на большое отрицательное значение, поэтому вероятность их выбора практически равна нулю.

$$l'_i = l_i, \text{ если } mask_i = 1; \quad l'_i = -10^9, \text{ если } mask_i = 0.$$

Маскирование является частью архитектуры принятия решения, а не только внешней проверкой. Оно резко сокращает эффективное пространство поиска и предотвращает расход обучающих итераций на заведомо невозможные действия.

5.4 Критик

Критик оценивает ожидаемую дисконтированную награду $V(s)$. На его вход поступает 64-мерный pooled battle embedding, после чего линейная value head $64 \rightarrow 1$ формирует скалярную оценку. При включённом centralized critic актёр продолжает использовать actor-centric observation, а критик получает отдельное полное наблюдение боя. Это позволяет улучшать оценку ценности без передачи политике скрытой информации.

Полная модель содержит 170 135 обучаемых параметров. По сравнению с крупными сверточными сетями это компактная архитектура, однако основная вычислительная нагрузка обучения дополнительно включает последовательную симуляцию правил боя и построение масок.

6 Обучение методом PPO

6.1 Сбор rollout

Trainer одновременно поддерживает несколько независимых CombatEnvironment. На каждом такте для активного участника строятся observation и action masks, модель выбирает действие, боевой движок выполняет его и возвращает reward. В rollout сохраняются actor observation, critic observation, компоненты действия, log probability, value, reward, terminal flag, маски, идентификатор среды и команда актёра.

Несколько сред позволяют пакетно выполнять прямой проход модели на GPU. Сама симуляция правил остаётся преимущественно CPU-задачей, поскольку содержит большое число ветвлений, проверок объектов Python и дискретных переходов состояния.

6.2 Оценка преимуществ GAE

Для снижения дисперсии градиентной оценки применяется Generalized Advantage Estimation. Сначала вычисляется temporal-difference ошибка, затем рекурсивно суммируются дисконтированные ошибки.

$$\delta_t = r_t + \gamma V(s_{t+1})(1 - d_t) - V(s_t),$$
$$\hat{A}_t = \delta_t + \gamma \lambda (1 - d_t) \hat{A}_{t+1}, \quad \hat{R}_t = \hat{A}_t + V(s_t).$$

В проекте используются $\gamma = 0,99$ и $\lambda = 0,95$. Преимущества нормализуются перед оптимизацией, что делает масштаб обновлений стабильнее при переходах между сценариями с различными наградами и длительностью.

6.3 Функция потерь

Основной компонент PPO ограничивает отношение вероятностей новой и старой политики. Это препятствует слишком резкому изменению поведения после одного rollout.

$$r_t(\theta) = \pi\theta(a_t/s_t) / \pi\theta_{old}(a_t/s_t),$$

$$L_{policy} = -E[\min(r_t \hat{A}_t, \text{clip}(r_t, 1-\varepsilon, 1+\varepsilon)\hat{A}_t)], \quad \varepsilon = 0,2.$$

Ошибка критика вычисляется как среднеквадратичное отклонение оценки $V(s)$ от рассчитанного return. Энтропийный член поощряет исследование и препятствует преждевременному переходу к почти детерминированной неэффективной политике.

$$L_{value} = \text{MSE}(V(s_t), \hat{R}_t),$$

$$L = L_{policy} + 0,5 \cdot L_{value} - 0,01 \cdot H(\pi).$$

Оптимизация выполняется Adam с шагом $3 \cdot 10^{-4}$. Для каждого rollout проводится четыре эпохи обновления по minibatch. Норма градиента ограничивается значением 0,5.

6.4 Гиперпараметры

Параметр	Значение	Назначение
learning_rate	0,0003	Шаг оптимизатора Adam.
gamma	0,99	Коэффициент дисконтирования.
gae_lambda	0,95	Сглаживание GAE.
clip_epsilon	0,2	Ограничение изменения политики PPO.
value_loss_coef	0,5	Вес ошибки критика.
entropy_coef	0,01	Вес энтропийной регуляризации.
max_grad_norm	0,5	Ограничение нормы градиента.
update_epochs	4	Число проходов по rollout.
rollout_steps	512 по умолчанию	Число переходов перед update; при экспериментах использовалось 1024.
max_episode_steps	256 минимум	Базовый лимит длительности эпизода.
steps_per_create	64	Масштабирование timeout по числу существ.

7 Формирование награды

Reward-функция является многоуровневой. Доминирующие сигналы соответствуют основной цели: победа и поражение имеют веса +5 и -5, убийство противника – +1. Дополнительные компоненты формируют тактическое поведение: нанесённый и полученный урон, сохранение союзников, полезные Grapple, Shove, Dodge, Disengage и Help, использование укрытий, эффективное расходование ресурсов, штрафы за friendly fire, иммунитеты, бесполезные предметы и затягивание боя.

Тактические коэффициенты намеренно существенно меньше терминальной награды. Благодаря этому агент не должен предпочитать локально выгодное укрытие завершению боя или избегать расхода заклинания, когда оно обеспечивает победу либо спасает союзника.

8 Curriculum learning и многосценарное обучение

Немедленное обучение на полном пространстве классов, заклинаний, предметов, карт и противников создаёт чрезмерно сложную исследовательскую задачу. Поэтому применяется curriculum learning из 13 уровней. Начальные этапы содержат дуэль Fighter против Goblin, затем добавляются Cleric, Orc, Wizard, AoE-заклинания, смешанные команды, дальний бой, сопротивления, препятствия, укрытия и сложная местность.

Для перехода используется окно из 20 завершённых эпизодов. Требуемый win rate равен 0,75, а минимальное время на уровне – 10 PPO updates. Текущий уровень сохраняется в checkpoint, поэтому обучение может продолжаться после перезапуска.

При обучении стороны игроков противники управляются role-aware aggressive baseline. Он выбирает стратегию ближнего боя, дистанционного удержания, лечения или применения заклинаний в зависимости от роли существа. Это создаёт стабильного активного противника и предотвращает

обучение на пассивном поведении. Поддерживаются также режимы обучения стороны врагов, общей политики и self-play с пулом предыдущих checkpoint.

9 Анализ архитектурных решений

9.1 Преимущества

- Переменное число участников обрабатывается через entity mask без изменения размерности параметров модели.
- Графовые рёбра явно описывают тактически значимые отношения, которые плоской MLP пришлось бы восстанавливать косвенно.
- Раздельный pooling союзников и противников сохраняет командную структуру боя.
- Иерархические головы соответствуют реальной структуре действий и не требуют гигантского плоского action space.
- Action masks гарантируют соблюдение правил и ускоряют исследование.
- Actor-critic разделяет выбор действия и оценку долгосрочной ценности состояния.
- Curriculum learning позволяет постепенно увеличивать число механик и сложность сценариев.

9.2 Ограничения

- В observation помещается не более четырёх ближайших союзников и четырёх противников; крупные бои требуют увеличения padding либо динамической обработки.
- Часть edge-признаков между неактивными существами аппроксимируется, поскольку observation строится относительно текущего актёра.

- Policy heads имеют фиксированные максимальные размерности, поэтому расширение контента требует контроля совместимости checkpoint.
- Суммирование log probability всех голов включает вспомогательные компоненты; корректность масок и sanitization индексов критична.
- Обучение зависит от качества reward shaping и распределения curriculum-сценариев; высокий win rate на одном уровне не гарантирует обобщение на следующий.
- Симуляция правил выполняется на CPU и может стать основным ограничением производительности даже при обучении модели на GPU.
- Имеющиеся логи отражают незавершённые эксперименты и не позволяют утверждать, что политика достигла окончательной сходимости на всех 13 стадиях.

9.3 Интерпретация метрик обучения

В отличие от обучения на одной стационарной выборке, средняя награда и win rate в curriculum-системе не обязаны монотонно возрастать. После перехода на более сложный уровень распределение состояний меняется, появляются новые действия и противники, поэтому метрики могут резко снизиться. Энтропия характеризует неопределённость политики: её постепенное уменьшение внутри одной стадии обычно указывает на формирование более устойчивого выбора. Однако слишком быстрое падение энтропии при низком win rate является признаком коллапса политики и недостаточного исследования.

Для корректной оценки следует строить графики отдельно по curriculum level либо наносить границы стадий на общую временную шкалу. Дополнительно необходимо учитывать долю timeout, число завершённых боёв, распределение действий и расход ресурсов.

10 Вывод

Разработанная архитектура объединяет структурированное представление D&D-подобного боя, графовую передачу сообщений и устойчивую оптимизацию PPO. GNN-кодировщик формирует контекст с учётом отношений между участниками, карта и глобальные признаки дополняют пространственную и сценарную информацию, а набор маскированных policy heads формирует только допустимые составные действия.

Архитектура соответствует текущей цели проекта – обучению тактического боевого агента на ограниченном подмножестве правил D&D-like 5e. Она допускает расширение состава персонажей и сценариев, но требует последовательного curriculum-обучения, контроля совместимости пространства действий и дальнейшей экспериментальной оценки на независимом наборе боевых ситуаций.